Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

ECEN 403

Midterm Presentation

Team 2: SICKSense
Paavan Bagla, Jason Agnew,
Samuel Ramos, Josiah Faircloth
TA: Vishwam
Sponsor: SICK

Project Summary

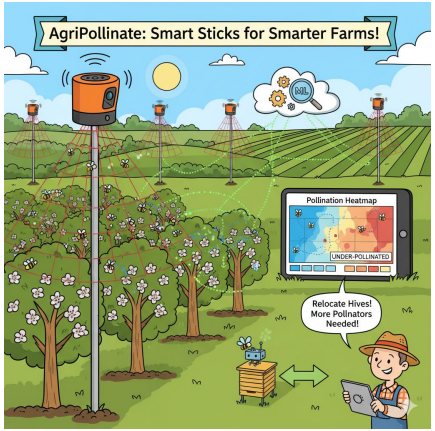
Problem statement:

- Farmers lack reliable, real-time insights into pollination activity. Existing methods are reactive, labor-intensive, and often inaccurate, leading to missed pollination deficits that directly reduce crop yield and quality.

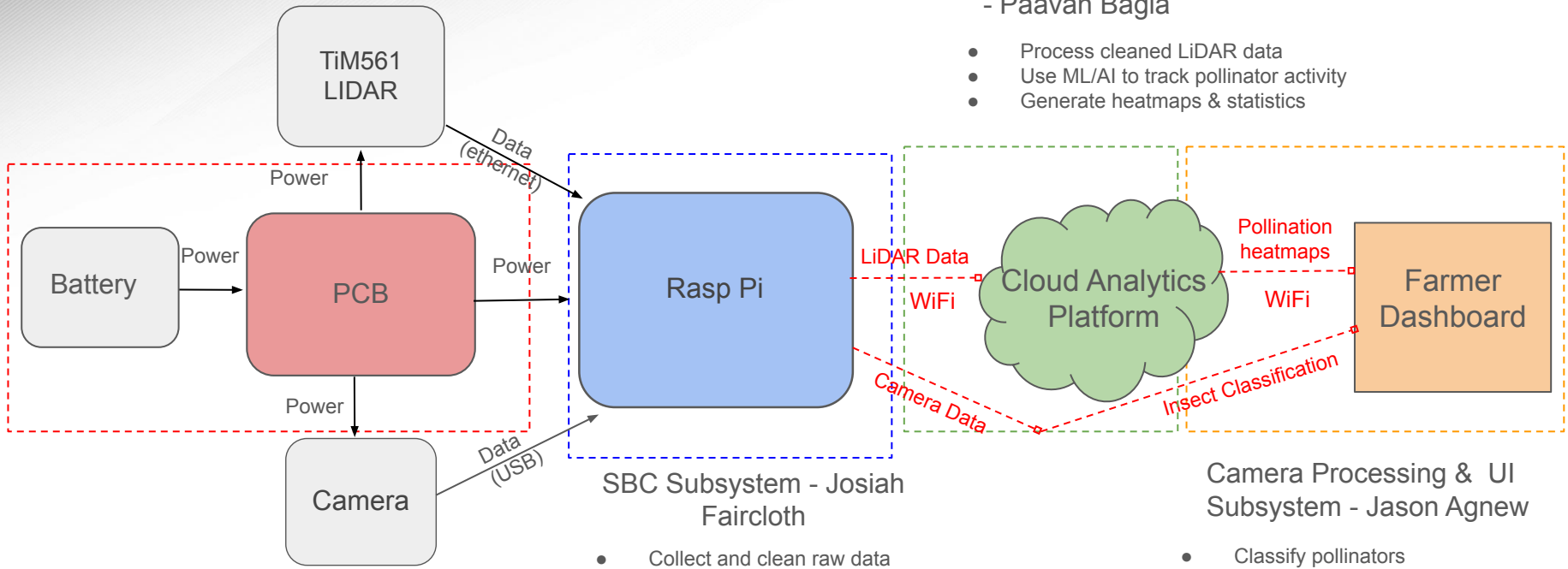
AgriPollinate

Our Solution:

- AgriPollinate is a Light Detection and Ranging (LiDAR)-powered monitoring system that continuously tracks pollinator activity and generates easy-to-read heatmaps.
- Farmers can access these heatmaps through a simple dashboard, allowing them to quickly identify under-pollinated areas, optimize hive placement, and improve crop yield and quality.



System Overview



LiDAR Processing Subsystem
- Paavan Bagla

- Process cleaned LiDAR data
- Use ML/AI to track pollinator activity
- Generate heatmaps & statistics

SBC Subsystem - Josiah Faircloth

- Collect and clean raw data
- Transmit cleaned data via WIFI

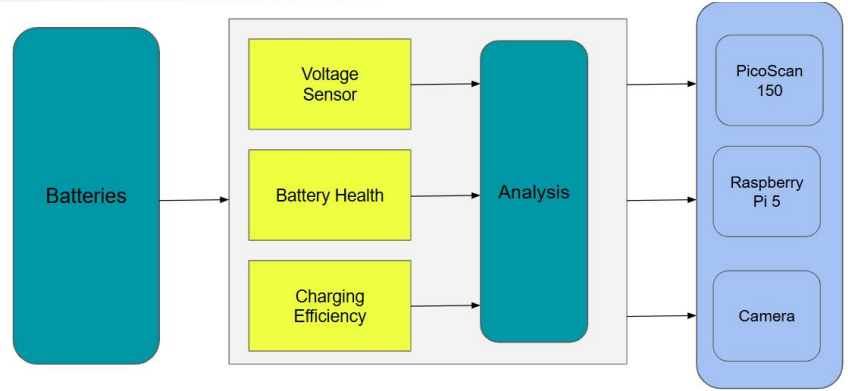
Camera Processing & UI Subsystem - Jason Agnew

- Classify pollinators
- Deliver heatmaps & statistics to easily digestible user interface

Power Subsystem - Samuel Ramos

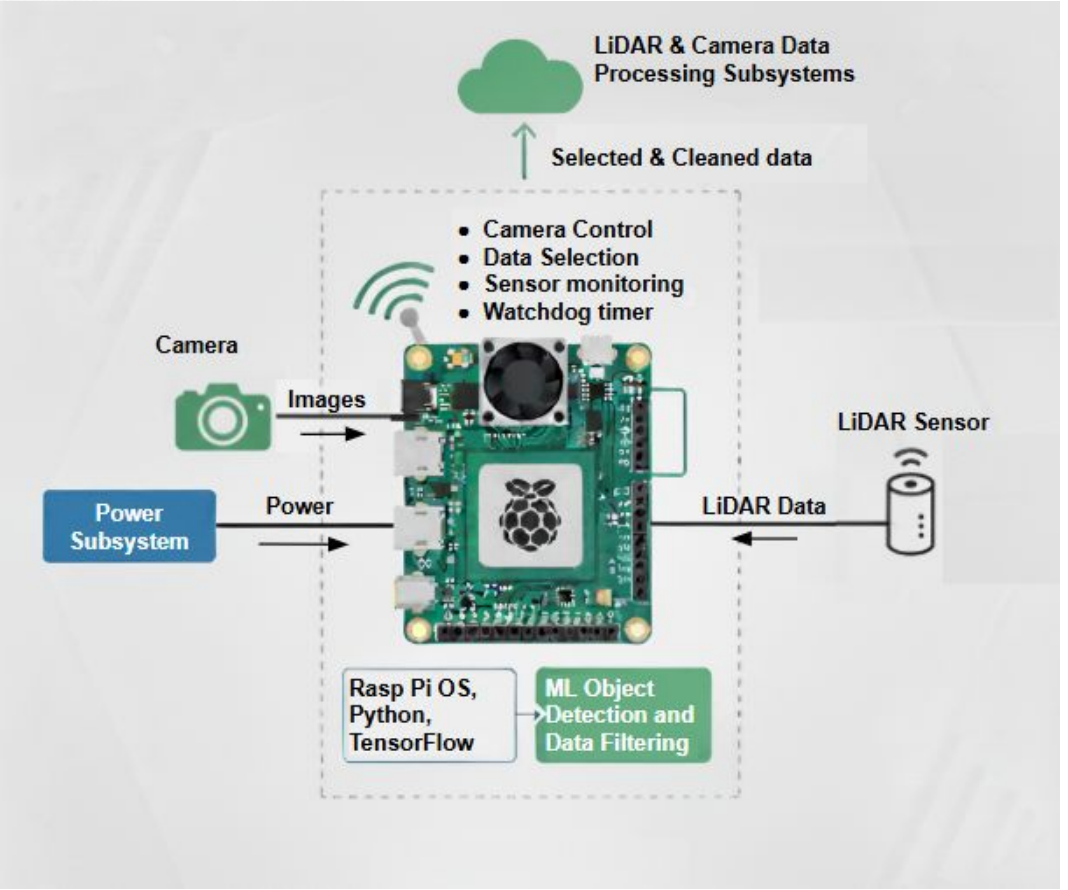
- Supply power to physical components
- Battery Management System

Battery Management System



- This subsystem is responsible for the safety of a rechargeable battery, which it does by monitors the health and charge of the battery.
- The BMS will be designed and tested using Altium.
- Objective: The battery is to remain operational, and later on improve on its efficiency.
- Constraints: Battery budget.
- Metrics: Charging Efficiency, Battery health, and Voltage range
- The BMS will connect the battery to the LiDAR, Camera, and Raspberry Pi.
- Powering each system while monitoring the battery.

Single Board Computer (SBC) Subsystem : Josiah Faircloth

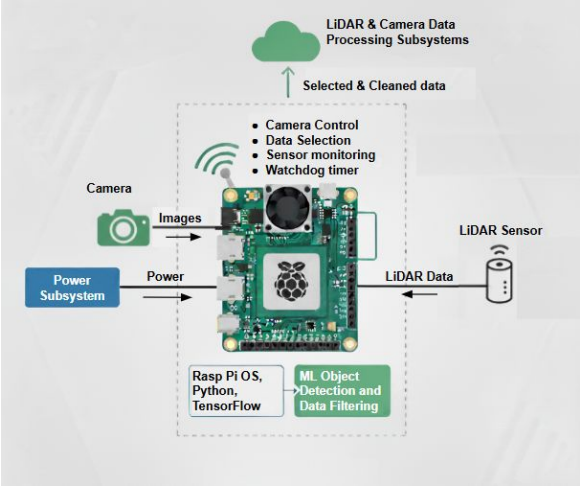


SBC System : Josiah Faircloth

- What does the subsystem do?
  - Collects and monitors data from LiDAR sensor and camera
  - Performs on-device machine learning for initial object detection and cleaning
  - Controls camera based on LiDAR Input

- How is it being designed?
  - **Tools:** Python, TensorFlow Lite, SOPAS ET, Raspberry Pi OS
  - **Key Parts:** Raspberry Pi 5

- Key objectives, constraints, metrics
  - **Objectives:** Reliably capture, process, and transmit data streams in real-time
  - **Constraints:** Power consumption, Thermal management (heatsink or fan)
  - **Metrics:** Inference Speed, Data Reduction

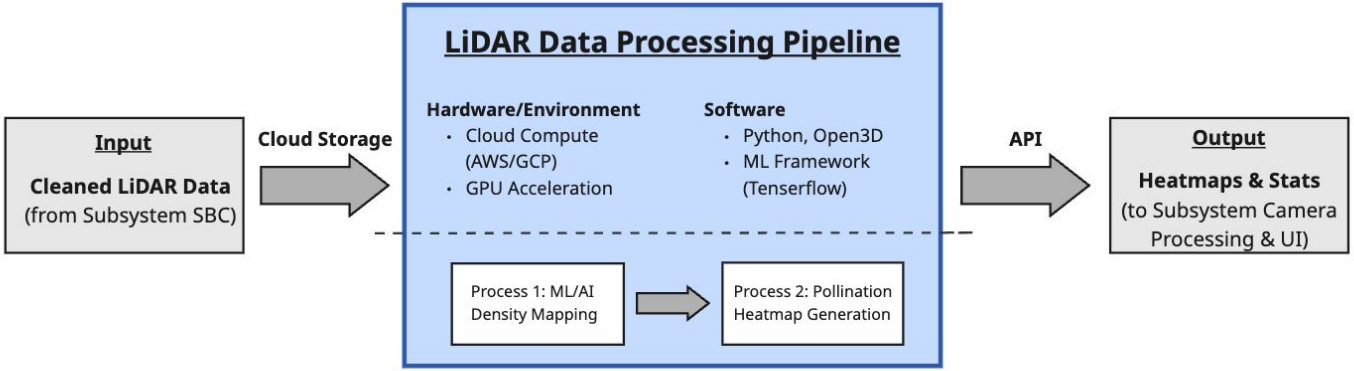


- Interact with other subsystems
  - **Input:** Direct connection to PicoScan150 (ethernet) and camera (USB), Powered by BMS (5V/5A USB-C)
  - **Output:** Transmits filtered LiDAR and camera data to cloud platform over WiFi

SBC Subsystem : Josiah Faircloth

Criteria	Python on Raspberry Pi	C++ / Arduino on ESP32
Capability	Full OS allows for complex software, multitasking, AI/ML	Bare metal. Limited heavy computation or multitasking
Development	Python has simple syntax and vast libraries	C++ can be complex. Fewer high-level libraries
Performance	Powerful hardware	C++ is efficient, but hardware is a limiting factor
Power Consumption	Requires a robust power solution	Can run for long periods on a small battery.

LiDAR Data Processing & Pollination Heatmaps System: Paavan Bagla



Key Objectives & Metrics

- Real-time, accurate insights
- Pollinator detection accuracy ~ 85%
- Heatmap refresh rate ~ 5 sec

Key Constraints & Dependencies

- Limited bandwidth, LiDAR data noise
- Dependency: Cleaned data from Subsystem SBC
- Dependency: Sync with Camera Processing & UI System

miro

LiDAR Data Processing & Pollination Heatmaps System: Paavan Bagla

- What does the subsystem do?
  - Processes cleaned LiDAR data from cloud storage.
  - Uses **Machine Learning (ML) / Artificial Intelligence (AI) models** to detect pollinator flight activity.
  - Generates **real-time heatmaps & statistics** showing pollinator density and under/over-pollinated zones.

- How is it being designed?
  - **Tools:** Python, TensorFlow, Open3D for point cloud processing.
  - **Pipeline:** LiDAR → Data cleaning (Subsystem SBC) → ML/AI density mapping → Dashboard visuals.
  - **Key Parts:** Cloud compute (Amazon Web Services (AWS) / Google Cloud Platform (GCP)), GPU acceleration.

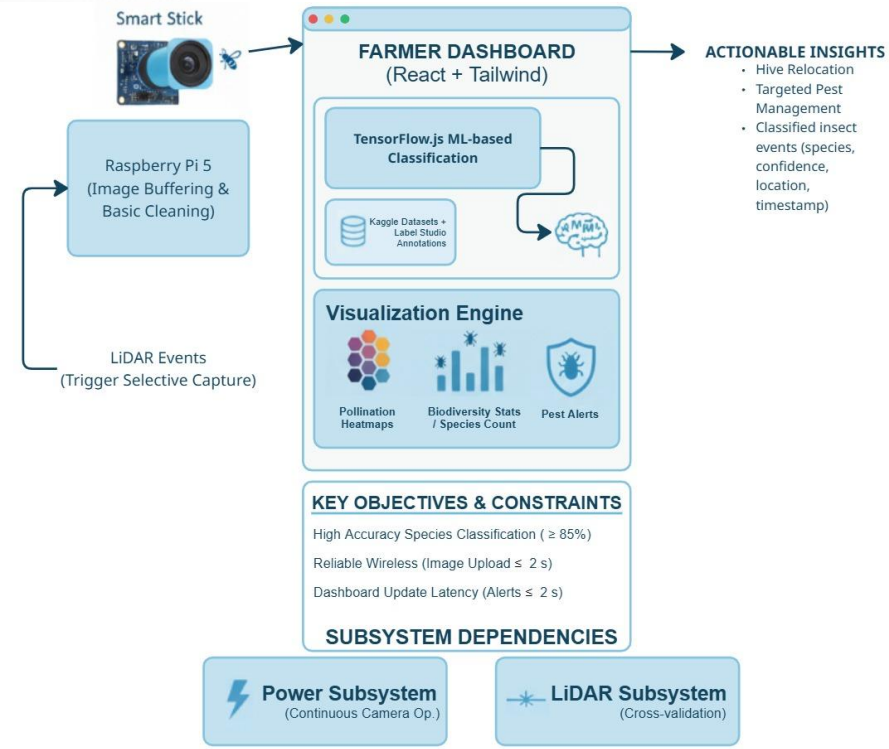
- Key objectives, constraints, metrics?
  - **Objectives:** Real-time, accurate pollination insights.
  - **Constraints:** Limited bandwidth, noise in LiDAR data, power limits.
  - **Metrics:** Pollinator detection accuracy ≥ 85%, heatmap refresh rate ≤ 5 sec.

LiDAR Data Processing & Pollination Heatmaps System : Paavan Bagla

- How will it connect to and interact with other subsystems?
  - **Input:** Cleaned LiDAR data from Subsystem SBC.
  - **Output:** Heatmaps & pollination density stats → Subsystem Camera Processing & UI System.
  - **Dependency:** Must sync with Camera-based classification for validation.

Option	Pros	Cons	Decision
TensorFlow	Stable, good deployment tools	Slightly heavier	Selected
PyTorch	Easier experimentation	Less optimized for deployment	Not Selected
Python	Rich libraries for ML and point cloud processing, strong community support	Slower runtime than C++ for heavy computation	Selected

Camera Processing & UI Subsystem: Jason Agnew



Camera Processing & UI Subsystem: Jason Agnew

- What does the subsystem do?
  - Captures insect images near smart sticks via camera modules.
  - Performs ML-based classification directly in the farmer dashboard using TensorFlow.js.
  - Dashboard visualizes results: pollination heatmaps, species count, pest alerts.
  - Provides farmers with actionable insights (hive relocation, pest management).

- How is it being designed?
  - **Tools:** Kaggle datasets, Label Studio for annotation, Tensorflow.js for in-browser ML, React + TypeScript + Tailwind for UI, Figma for design.
  - **Pipeline:** Camera → Rasp Pi 5 (buffer + basic cleaning) → Dashboard (classification + visualization)
  - **Key Parts:** Pi HQ Camera, Raspberry Pi 5 or similar for image forwarding

- Key objectives, constraints, metrics?
  - **Objectives:** High-accuracy species classification for pollinators vs pests, seamless real-time farmer dashboard with intuitive visuals.
  - **Constraints:** Requires reliable wireless connection, bandwidth management (large image/video files), Latency must be low enough for “real-time” alerts.
  - **Metrics:** Classification accuracy ≥ 85%, image upload latency ≤ 2 sec, dashboard update latency ≤ 10 sec.

Camera Processing & UI Subsystem: Jason Agnew

- How will it connect to and interact with other subsystems?
  - Inputs:
    - Camera images from SBC
    - LiDAR events to trigger selective image capture
  - Outputs:
    - Classified insect events (species, confidence, location, timestamp)
    - Dashboard visualizations: pollination heatmaps, biodiversity stats, pest alerts.
  - Dependency:
    - Relies on power subsystem for continuous camera operation.
    - Integrates with LiDAR subsystem for cross-validation of pollinator activity.

Criteria	TensorFlow.js	TensorFlow on Cloud
Capability	Runs directly in browser.	Full TensorFlow stack.
Development	Easy integration with web apps. JavaScript ecosystem.	Integrates with enterprise ML pipelines, APIs, and cloud storage. Python ecosystem.
Performance	Limited by client hardware (CPU/GPU in browser).	Access to powerful GPUs.
Scalability	Each user runs locally, no central server load.	Can scale to millions of users via APIs

Project Execution & Validation

	9/30	10/07	10/21	10/28	11/11	11/25	12/02
BMS (Samuel Ramos)	Familiarize myself with Altium, began working on PCB design.	Continue working on PCB, began preliminary testing	Finished PCB Design and already have it submitted to be made.	Develop testing plans for the BMS & see if any improvements can be made.	Receive PCB and begin hardware testing.	Full systems test and make revision notes.	Prepare report on BMS status and proposed improvements.
SBC Subsystem (Josiah Faircloth)	Research & Finalize SBC choice, Install needed libraries and development environments.	Procure SBC, LiDAR, camera, Install OS and establish remote access. **Validation**: SBC is booting and accessible for remote development via SSH.	Interface camera module with SBC, Develop script to capture images, Establish data pipeline to cloud. **Validation**: Camera captures an image within 200ms of a “detection”.	Interface LiDAR by configuring IPs, Develop script to parse data stream. **Validation**: Script connects to LiDAR, captures data stream, parses 5 mins of point cloud data.	Develop software to filter raw LiDAR data, Implement ML model to distinguish potential insects from noise. **Validation**: System can process continuous LiDAR stream and output coordinates for a simulated "object" with <5% false pos/neg	Integrate LiDAR processing, camera capture, and cloud transmission, Implement watchdog timer. **Validation**: Operates for 12 hours w/o intervention, triggers camera on LiDAR detections, transmits data. Watchdog timer is functional	Optimize code for efficiency, Prepare live demo of full functionality. **Validation**: Operates with >98% data transmission success rate and correctly triggers the camera for >90% of simulated targets

Project Execution & Validation

	9/30	10/07	10/21	10/28	11/11	11/25	12/02
LiDAR Data Processing (Paavan Bagla)	Literature review on LiDAR applications in pollinator monitoring. Identified at least 3 LiDAR datasets for evaluation.	Select dataset (based on quality & resolution); Initial preprocessing scripts written (Python) for formatting LiDAR point clouds.	ML environment set up (Python + TensorFlow). Baseline model pipeline implemented and able to accept cleaned LiDAR data.	First round of model training completed. Generate and visualize initial density maps from small dataset subset.	Model refinement, incorporate filtering for noise reduction. Accuracy tested against validation subset; document metrics.	Generate refined pollination density heatmaps from dataset. Document preliminary results and limitations (accuracy, clarity of heatmaps).	Deliver semester prototype: dataset-driven pollination heatmap (proof-of-concept). Summarize dataset quality, model performance, and integration plan.
Camera Processing & UI Subsystem (Jason Agnew)	Finalize subsystem scope, set up repos for the dashboard and ML, and secure insect datasets. **Validation**: repository initialized and at least one dataset successfully loaded for inspection.	Complete initial Figma designs for the farmer dashboard and begin data annotation in Label Studio. **Validation**: mockups are ready for review and at least 200 images are labeled.	Train a baseline ML classifier, export it for TensorFlow.js, and create a dashboard component to display classification results with mocked data. **Validation**: model reaches ≥60% accuracy and dashboard displays test classifications.	Refine the ML model with more training and preprocessing, and add visualization features like heatmaps and species counts. **Validation**: model achieves ≥70% accuracy and heatmaps render correctly with simulated inputs.	Simulate the image pipeline (as if from Raspberry Pi) and test dashboard latency for real-time updates. **Validation**: dashboard refreshes within ≤10 sec and simulated pest/pollinator alerts appear correctly.	Integrate simulated LiDAR triggers for cross-validation and stress test the system with multiple events. **Validation**: classification accuracy reaches ≥80% and the system handles sequential inputs without failure.	Deliver a full subsystem demo showing dataset → classification → visualization → alerts, and document setup for next year's real Pi integration. **Validation**: classification accuracy reaches ≥85% and final documentation/demo are completed.

Questions?